

Copy Semantics

Copy semantics defines the exact rules a programming language, or a specific object, follows when data is duplicated. "**Shallow vs. Deep copy**" describes the results. Semantics define the mechanism and default behavior of copying.

Copy Operation is performed on **3** places.

1. When using the "=" operator,
2. When passing a **variable/object** to a function,
3. When returning a **variable/object** from a function.

In **2 and 3**, the original **variable/object** is destroyed, and C++ creates a **brand-new copy** to use inside the function and a **brand-new copy** to return. While the copy occurs in any of the 3 places, Copy semantics dictate what actually happens in memory.

Why copy instead of using the existing one?

It's all about **Scope and the Stack**.

In C++, when a function finishes, its "**stack frame**" is wiped clean. Any object created inside that function (that isn't on the heap) **dies immediately**.

The Problem: Scope Death: Imagine a function that creates a Car:

```
Car getCar() {  
    Car localCar; // This lives on the Stack  
    return localCar;  
} // <-- localCar is DESTROYED here!
```

If you didn't create a new object (via copy or move), the calling function would be left holding a reference to a "dead" object (memory that has already been cleared)

Why can't we just "use the existing one":

1. **Memory Security:** The local variable's memory address is no longer valid once the function returns. Using it would cause a crash.
2. **Ownership:** The new object in the calling function "**takes over**" the data so it can live on in the new scope.

Because of this, it creates inefficiency for large amounts of data, such as a vector array holding **100,000** integers on the heap. Copying it just to return to the outer scope is a huge waste. This problem is solved using a concept called "**Move Constructor.**" (Will Study Next)

1. Value Semantics (Pass-by-Value)

Guarantees that a completely independent, deep clone of the data is created upon assignment.

- **The Mechanism:** The actual bits of data are duplicated into a new memory location.
- **The Result:** Total isolation. Mutating the copy will never affect the original.
- **When it happens:** Most languages use this for primitive types (ints, bools, floats).

2. Reference Semantics (Pass-by-Reference)

When reference semantics are used, the actual data is never duplicated. Instead, the language quietly passes the memory address (the pointer).

- **The Mechanism:** Only the pointer is copied. You now have two variables pointing to the exact same block of Heap memory.
- **The Result:** Shared state. Mutating the copy modifies the original.
- **When it happens:** Many languages default to this for complex data structures (objects, arrays, classes) to save memory and CPU cycles.

How Copy Semantics Work in Practice

Different languages have entirely different default copy semantics.

JavaScript / TypeScript: JavaScript strictly divides its copy semantics based on the data type:

- **Primitives** (Number, String, Boolean) use strict **Value Semantics**.
- **Objects, Arrays, and Functions** use strict **Reference Semantics**.

Go (Golang): In Go, absolutely everything is passed by value. Period. However, it often feels like **reference semantics** when you pass a **Slice** or a **Map**. **Why? Because**

- When you pass a slice to a function, Go does a strict pass-by-value copy of that struct. And copies the **pointer**, the **length**, and the **capacity**.
- Because the copied pointer still points to the exact same underlying array in memory, mutating the slice elements inside the function will mutate the original array.

C++ (Manual Copy Semantics): In C++, we are the architect. If we create a custom class, C++ will do a shallow copy by default. But because you manage memory manually (**new/delete**), a shallow copy is incredibly dangerous. If two objects share a pointer to the same Heap data, and both try to delete it when they die, your program will crash (a "Double Free" error).

To fix this, C++ allows you to define **your own copy semantics** by writing a **Copy Constructor**. This is a special method where you explicitly tell the compiler,

"When someone tries to copy this object, do not just copy the pointer. Go to the Heap, allocate fresh memory, and do a true deep copy."

C++ and Go both use value semantics, but Go doesn't have a copy constructor

Deep Copy vs. Shallow Copy

1. Shallow Copy (Sharing the Inside)

A shallow copy creates a new top-level object, but it does **not** recursively clone nested objects. Instead, for any property that is a **reference type**, it simply copies the memory address (the pointer).

- **The Concept:** You get a new outer box, but the items inside the box are just pointers to the exact same items as the original box.
- **The Danger:** Because both the original and the copy point to the same nested data in the Heap, modifying a nested property in the copy **will mutate the original object as well**.

Analogy: Sharing a link to a Google Doc. You both have your own distinct link (top-level), but if either of you edits the document, the other sees the changes because you are looking at the same underlying file.

Note: Shallow copy behaves differently in different languages. **(Copy Semantics)**

2. Deep Copy (Total Isolation)

A deep copy creates a completely independent clone of the original object, along with brand new copies for any property that is a **reference type**.

- **The Concept:** It doesn't copy the pointer but allocates a new space in the heap and copies the actual data present at the memory address to the new allocated space.
- **The Benefit:** Total data isolation. You can completely destroy or mutate the copied object, and the original object remains perfectly untouched.

Analogy: Downloading a Google Doc as a PDF and emailing it to someone. They have a completely separate, disconnected file on their hard drive.